Alberto Andrés Valdés González.
Degree: Mathematical Engineer.
Work position: ML-Engineer.
Mail: anvaldes@uc.cl/alberto.valdes.gonzalez.96@gmail.com
Location: Santiago, Chile.

LLM as a Judge: rules for a reliable evaluator

Setting: We have a dataset of samples $x_1, x_2, \ldots, x_N$. Each sample already carries an **old label** y_i^{old}, produced by an agent, a heuristic or a previous annotator. An **LLM Judge** J reviews every sample and returns the **true label** y_i^* together with a justification:

$$J(x_i,\ y_i^{old},\ \text{rubric}) \longrightarrow \left(\ y_i^*,\ \ v_i \in \{\text{correct}, \text{incorrect}\},\ \ r_i^{old},\ \ r_i^*\ \right)$$

where v_i says whether the old label is right, r_i^{old} explains *why* the old label is correct or incorrect, and r_i^* explains *why* the true label is the right one.

Goal: the judge is not a second agent. It is an **independent auditor**: its errors must not move together with the errors of the agent it evaluates.

Pipeline:

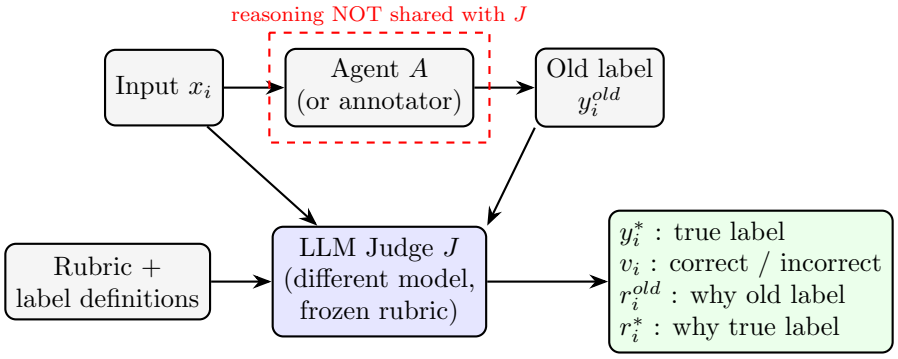


Figure 1: The judge sees the input, the rubric and the old label, but never the agent's reasoning, confidence or identity.

Rule 1: always expose the true label.

The judge must return an explicit label y_i^* from the closed label set $\mathcal{Y}$ (plus an optional `UNCERTAIN` option). A verdict like "the old label is wrong" without saying which label is right is useless: it cannot be used to correct the dataset nor to measure the agent.

$$y_i^* \in \mathcal{Y} \cup \{\texttt{UNCERTAIN}\}, \qquad v_i = \begin{cases} \text{correct} & \text{if } y_i^* = y_i^{old} \\ \text{incorrect} & \text{if } y_i^* \neq y_i^{old} \end{cases}$$

Rule 2: justify the old label (correct or incorrect).

The field r_i^{old} must explain, using evidence from x_i and the rubric, *why* the old label holds or fails:

- If v_i = correct: which part of the input supports the old label and which rubric definition it satisfies.
- If v_i = incorrect: which rubric condition the old label violates, or which evidence it ignored (e.g. a plausible but superficial keyword match).

Rule 3: justify the new (true) label.

The field r_i^* must show why y_i^* is the right one: cite the concrete evidence in x_i and map it to the definition of y_i^* in the rubric. When $y_i^* \neq y_i^{old}$, it should also explain why y_i^* fits *better* than y_i^{old}, not only that it fits.

Rule 4: the judge must not be correlated with the agent.

Let $e_i^A = \mathbf{1}[y_i^{old} \neq y_i^{true}]$ be the agent's error and $e_i^J = \mathbf{1}[y_i^* \neq y_i^{true}]$ the judge's error (measured on a human gold set). We want the errors to be (approximately) independent:

$$P(e^J = 1 \mid e^A = 1) \;\approx\; P(e^J = 1) \qquad \Longleftrightarrow \qquad Corr(e^A, e^J) \approx 0$$

If the judge fails exactly where the agent fails, it will **confirm** the agent's mistakes and the measured accuracy will be inflated. In practice:

(a) **Different model:** use a different model (ideally a different family) than the agent, never the same checkpoint with another prompt. This reduces shared blind spots and self-preference bias.

(b) **Different prompt and context:** the judge must not reuse the agent's system prompt, few-shot examples or retrieved context.

(c) **Blind to the agent:** the judge does not see the agent's chain of thought, its confidence score, nor which agent or version produced the label.

(d) **Label first, compare later (anti-anchoring):** the judge first decides y_i^* from x_i and the rubric, and only then reads y_i^{old} to write v_i, r_i^{old} and r_i^*. This avoids simply agreeing with whatever label is shown.

(e) **Alternative: use the old label as input (verify, not infer):** option (d) is valid, but it is equally valid to give y_i^{old} to the judge from the start and ask it to *verify* it. An LLM usually **compares better than it infers**: checking whether a given label satisfies the rubric is an easier task than producing the label from scratch, so the judge can be more accurate in this mode. The price is the risk of anchoring (the judge tends to agree with the label shown). To control it:

- ask the judge to first look for evidence *against* y_i^{old} and to name the strongest alternative label before deciding;

- measure on the gold set how often the judge catches a wrong old label, $P(v = \text{incorrect} \mid y^{old} \neq y^{true})$, not only its global accuracy;
- compare both modes, (d) and (e), on the same gold set and keep the one with the best agreement with humans.

(f) **No Goodhart:** the agent must not be tuned by optimizing directly against this same judge. If it is, keep a separate, held-out judge (or human set) for the final evaluation.

Rule 5: frozen, explicit rubric.

Every label in $\mathcal{Y}$ has a written definition, inclusion / exclusion criteria and a tie-breaking rule. The rubric and the judge prompt are versioned; the judge runs with temperature 0, so the same input gives the same verdict.

Rule 6: evidence over opinion.

Every justification must quote or point to concrete spans of x_i. Statements not grounded in the input ("it seems more likely that…") are not accepted. If the evidence is insufficient, the judge returns `UNCERTAIN` instead of guessing, and the sample goes to human review.

Rule 7: control known biases.

Length / verbosity bias (preferring long answers), position bias (preferring the first option in pairwise comparisons) and format bias. For pairwise evaluation, run both orders (a, b) and (b, a) and keep the verdict only if it is consistent.

Rule 8: validate the judge itself (meta-evaluation).

Before trusting the judge, compare it against a human-labeled gold set of size M:

$$Acc(J) = \frac{1}{M} \sum_{i=1}^{M} \mathbf{1}[y_i^* = y_i^{human}], \qquad \kappa = \frac{p_o - p_e}{1 - p_e}$$

where p_o is the observed agreement and p_e the agreement expected by chance (Cohen's kappa). The judge is accepted only if its agreement with humans is close to the agreement between two humans. Report it per class: a judge can be good on average and bad on the rare classes.

Output format. The judge always answers with a fixed, parseable schema:

```
{
  "true_label": "BILLING",
  "old_label": "TECHNICAL_ISSUE",
  "old_label_verdict": "incorrect",
  "why_old_label": "No malfunction, error or bug is described. The user does
     not report that the product fails; the rubric requires a failure of the
     service for TECHNICAL_ISSUE.",
  "why_true_label": "The user reports a double charge ('charged twice') and
     asks for a refund. The rubric defines BILLING as charges, invoices or
     refunds. BILLING fits better than TECHNICAL_ISSUE because the problem is
     about money, not about the service working.",
  "evidence": ["charged twice for my subscription", "please refund one of the
     payments"],
  "confidence": "high"
}
```

Example 1 (old label incorrect):

Input: "I was charged twice for my subscription this month, please refund one of the payments."
Old label: `TECHNICAL_ISSUE` ⇒ *True label:* `BILLING`.

- **Why the old label is incorrect:** the text contains no failure of the service; the agent probably reacted to the word "subscription" as a product issue.
- **Why the new label is okay:** "charged twice" and "refund" match the rubric definition of `BILLING` directly.

Example 2 (old label correct):

Input: "The app crashes every time I open the payments screen."
Old label: `TECHNICAL_ISSUE` ⇒ *True label:* `TECHNICAL_ISSUE`.

- **Why the old label is correct:** "crashes every time" is a malfunction of the service, which is exactly the rubric condition.
- **Why the true label is okay (and not `BILLING`):** the word "payments" only locates the screen; there is no charge, invoice or refund in dispute.

Judge prompt (template):

```
You are an independent evaluator. You did NOT produce the label below.
Label set: {LABELS}. Definitions and tie-break rules: {RUBRIC}.

Step 1. Read the INPUT and decide the true label using only the rubric.
        If the evidence is insufficient, answer UNCERTAIN.
Step 2. Now read the OLD LABEL. Say if it is correct or incorrect.
Step 3. Explain why the old label is correct/incorrect, citing the input.
Step 4. Explain why the true label is correct, citing the input,
        and why it fits better than the old label if they differ.
Return ONLY the JSON schema: {SCHEMA}.

INPUT: {x}
OLD LABEL: {y_old}
```

Summary:

A good LLM Judge (1) always exposes the true label, (2) explains why the old label is correct or incorrect, (3) explains why the new label is the right one, with evidence, and (4) is independent of the agent: different model, different prompt, blind to the agent's reasoning, either label-first to avoid anchoring or verifying the old label (LLMs compare better than they infer) with anchoring controls, and never the same judge the agent was optimized against. Its own reliability is measured against humans before its verdicts are trusted.
